Time Complexity Study Plan

30 Essential Asymptotic Analysis Problems & Solutions

PHASE 1: ITERATIVE & BASIC LOOPS

Q1. [O(n)]

A straightforward linear loop that runs exactly n times.

Q2. [O(n²)]

Two nested loops, where both the outer and inner loops execute n times ($n \times n = n^2$).

Q3. [O(n²)]

Dependent nested loops. Total iterations form an arithmetic progression: $1 + 2 + ... + n = n(n+1)/2$, which simplifies asymptotically to $O(n^2)$.

Q4. [O(log n)]

The loop control variable i doubles each iteration, reaching n in logarithmic steps.

Q5. [O(log n)]

The loop control variable starts at n and is halved during each iteration until it reaches 1.

Q6. [O(n log n)]

The outer loop runs n times, while the inner loop runs log n times. Combining them yields O(n log n).

Q7. [O(n)]

The variable decreases linearly by 1 each iteration, resulting in n total steps.

Q8. [O(log n)]

The value of n is divided by 2 continuously until it drops to or below 1.

Q9. [O(n)]

Two independent, sequential loops each running O(n) times. Total cost is O(2n), which drops constants to become O(n).

Q10. [O(n²)]

Sequential blocks of O(n) and $O(n^2)$. The asymptotic dominance rule keeps only the highest order term: $O(n^2)$.

PHASE 2: ADVANCED LOOPS & POLYNOMIAL GROWTH

Q11. [O(n³)]

Multiplicative nested execution layers of n and n^2, leading to $n \times n^2 = n^3$.

Q12. [O(log n)]

The loop variable grows exponentially by tripling each time ($O(\log_3 n)$). By the base-change formula, this is asymptotically equivalent to standard O(log n).

Q13. [$O(n^3)$]

The inner loop runs up to i^2 times. Summing this across all iterations gives $\Sigma(i^2) = n(n+1)(2n+1)/6$, which is a cubic polynomial of degree 3.

PHASE 3: RECURSIVE ALGORITHMS

Q14. [O(n)]

A single linear recursive call that decrements the problem size by 1 (n-1) at each stack level.

Q15. [O(log n)]

A recursive call where the problem size is halved (n/2) each time, identical to binary search.

Q16. [$O(2^n)$]

The function branches into two recursive calls at each level of a tree with depth n, generating exponential growth.

Q17. [$O(2^n)$]

Classic naive/un-memoized Fibonacci recursion. Forms a binary recursion tree with a bounding complexity of roughly $O(1.618^n)$, bounded as $O(2^n)$.

Q18. [O(n)]

A recursion structure that preserves a linear execution path of one frame per depth level.

Q19. [O(n)]

The function executes sequentially exactly n times through single-step recursive reduction.

Q20. [O(log n)]

The input parameter drops logarithmically as it divides by 2 on every recursive iteration.

PHASE 4: RECURRENCE RELATIONS & MASTER THEOREM

ID	Recurrence Relation	Complexity	Explanation / Breakdown
Q21	T(n) = T(n-1) + 1	**O(n)**	Solving via expansion gives 1+1+...+1 = n steps.
Q22	T(n) = T(n-1) + n	**$O(n^2)$**	Expands out to the summation of first n integers: 1+2+...+n = $O(n^2)$.
Q23	T(n) = T(n/2) + 1	**O(log n)**	Governed by Case 2 of Master Theorem (or substitution), matching binary search.
Q24		**$O(2^n)$**	

ID	Recurrence Relation	Complexity	Explanation / Breakdown
	T(n) = 2T(n-1) + 1		Each reduction doubles the work density, yielding a geometric series sum of $2^n - 1$.
Q25	T(n) = T(n/2) + n	**O(n)**	Master Theorem Case 3: the linear work done at the root dominates the total recursion cost.

PHASE 5: COMPLEX LOOPS & MIXED RELATIONS

Q26. [O(n log n)]

The inner loop acts multiplicatively up to the outer limit counter i, totaling an integrated summation that simplifies to O(n log n).

Q27. [O(n log n)]

The outer framework scales down logarithmically while driving an inner operational loop of static scale n.

Q28. [O(n^2)]

Total operation overhead decreases sequentially by single unit steps: n + (n-1) + ... + 1 = O(n^2).

Q29. [O(n log n)]

Standard combination structure: a linear outer pass running n instances of a logarithmic inner control sequence.

Q30. [O(n)]

Recurrence: T(n) = 2T(n/2) + O(1). Master Theorem Case 1 applies, where work is bound evenly by the leaf count of the tree.